AVANT

dino_game.cpp

```
g_Pos
g_bleft
g_hitTime

DinoGame_Init()

DinoGame_Update()

DinoGame_Shut()
```

VAR GLOBALES

FONCTIONS LIBRES

APRÈS

dino_player.h/.cpp

```
class DinoPlayer {
    m_pos
    m_bleft
    m_hitTime

public:
    Init()
    Update()
    Shut()
}
```

VAR MEMBRES

FONCTIONS MEMBRES = MÉTHODES

dino_game.cpp

```
vector<DinoPlayer> g_Players;
for (DinoPlayer& p: g_Players)
    p.Update()
```

FOR-RANGE LOOP

INSTANCE PASSÉE IMPLICITEMENT

SYNTAXE C++ (Fonction Membre)

```
void DinoPlayer::Update( ———— ) {
    m_pos += SPEED x t;
```

<=> this -> m_pos += SPEED x t;

```
}
```

Le compilateur ajoute implicitement un argument de fonction : l'instance

SYNTAXE C (Fonction libre)

```
void DinoPlayer_Update(DinoPlayer * p, ———— ){
    p -> m_pos += SPEED x t;
}
```

Avantages syntaxe C++

```
for (DinoPlayer & p : g_Players)
    p.Update(——)
    DinoPlayer_Update(p, ——)
```

C++ / C } 1. Simplicité d'écriture

```
class DinoPlayer {
public:
    |
private:
    |
};
```

← Accessible partout

← Accessible que dans les fonctions membres = méthodes

Fonction libre

Fonction membre

2. Permet de ne pas tout mettre en public

dino-player.h / .cpp

```
class DinoPlayer {
public:
    MÉTHODES

private:
    VAR MEMBRES
};
```

Reste de la base de code

Si variable incohérente (bug)
Alors besoin d'investiguer que les méthodes

Ex: Sur Flight Simulator 2024, il y a ~20 000 .h et ~15 000 .cpp
⇒ Très utile de garantir que pour résoudre un bug, il n'y a pas besoin de lire toute la base de code

Les méthodes doivent garantir que les variables membres restent cohérentes.

→ « Les méthodes doivent préserver les <u>invariants</u> de la classe »

Invariants = Règles qui dictent que les variables membres sont cohérentes.

Exemples d'invariants :

→ Un DinoPlayer a <u>toujours</u> un textureID valide.

→ Un DinoAnimal est <u>toujours</u> sur le terrain.

⟹ ENCAPSULATION

Autre avantage de l'encapsulation

dino-player.h/.cpp

```
class DinoPlayer {
public:
    MÉTHODES

private:
    VAR MEMBRES
};
```

↳ si besoin de changer les var membres (nouvelle feature...), juste besoin de changer les méthodes

Reste de la base de code

struct / class en C++

→ En C++, il n'y a pas de diff fondamentale entre les deux

```
struct T {
    int a;        ← Public
public:
    |
private:
    |
};
```

```
class T {
    int a;        ← Private
public:
    |
private:
    |
};
```

struct / class en C++

→ En C++, il n'y a pas de diff fondamentale entre les deux

→ Mais, par convention, on préfère utiliser ces mots-clés différement :

struct

→ Amas de données

DinoVec2 : (x, y)

DinoVertex : (x, y / u, v / color)

→ Pas de cohérence à maintenir

→ Pas d'invariants

→ Pas besoin de "private"

class

→ Logique de comportement

DinoPlayer :
- Il bouge avec la manette
- On peut le dessiner
- On peut le paralyser...

→ On se fiche des données utilisées pour représenter ce comportement

→ Tout passe par les méthodes

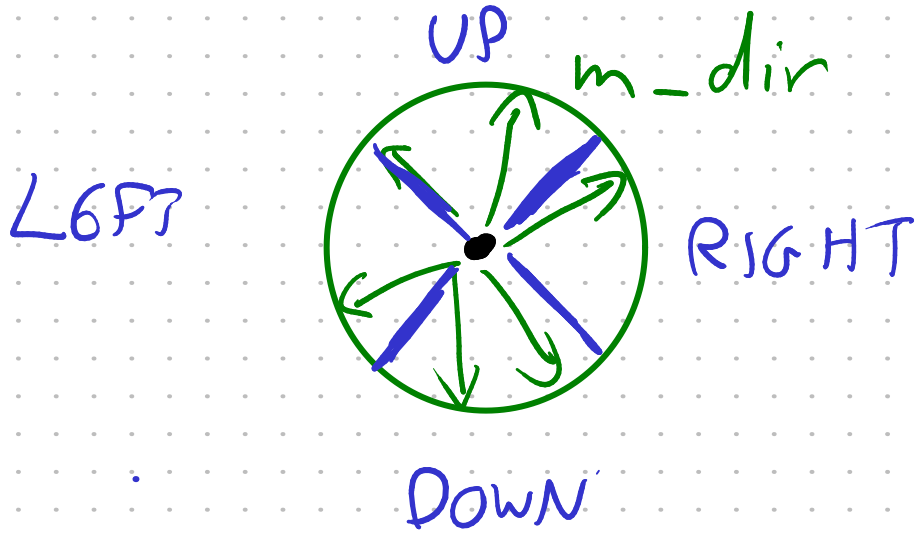
→ Encapsulation public / private

5b)

XDino_RandomUnitVec2()

⇒ Vecteur unitaire : sa longueur vaut 1

⇒ $\sqrt{x^2+y^2} = 1 \iff x^2+y^2 = 1$



Mouvement horizontal :

$|dir.x| > |dir.y|$

Mouvement vertical

$|dir.x| < |dir.y|$

CoursProgJV\ *.cpp|*.h

Nom	Chemin
pix3.h	C:\Users\j.vernay\Documents\CoursProgJV\external\pix\Include\WinPixEventRuntime
pix3_win.h	C:\Users\j.vernay\Documents\CoursProgJV\external\pix\Include\WinPixEventRuntime
PIXEvents.h	C:\Users\j.vernay\Documents\CoursProgJV\external\pix\Include\WinPixEventRuntime
PIXEventsCommon.h	C:\Users\j.vernay\Documents\CoursProgJV\external\pix\Include\WinPixEventRuntime
PIXEventsLegacy.h	C:\Users\j.vernay\Documents\CoursProgJV\external\pix\Include\WinPixEventRuntime
sokol_app.h	C:\Users\j.vernay\Documents\CoursProgJV\external\sokol
sokol_gfx.h	C:\Users\j.vernay\Documents\CoursProgJV\external\sokol
sokol_glue.h	C:\Users\j.vernay\Documents\CoursProgJV\external\sokol
sokol_log.h	C:\Users\j.vernay\Documents\CoursProgJV\external\sokol
sokol_time.h	C:\Users\j.vernay\Documents\CoursProgJV\external\sokol
stb.cpp	C:\Users\j.vernay\Documents\CoursProgJV\external\stb
stb_image.h	C:\Users\j.vernay\Documents\CoursProgJV\external\stb
dino_animal.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_animal.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_draw_utils.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_draw_utils.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_game.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_geometry.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_geometry.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_player.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_player.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_terrain.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_terrain.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
xdino.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
xdino_impl.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
xdino_linux_main.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-linux
xdinordr.sokol.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-linux
xdino_win64_main.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-windows
xdino_win64_rdr.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-windows
xdino_win64_rdr.h	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-windows

external\pix } Bibliothèque tierc

external\sokol } AIDE POUR LINUX

external\stb → PNG

} CODE EXTERNE

src\dino } GAMEPLAY

src\dino\x64-linux

src\dino\x64-windows

} MOTEUR PLATEFORME

CoursProgJV\ *.cpp|*.obj - Everything

Fichier Edition Affichage Recherche Signets Outils Aide

CoursProgJV\ *.cpp|*.obj

Nom	Chemin
stb.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_External
dino_animal.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
dino_draw_utils.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
dino_game.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
dino_geometry.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
dino_player.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
dino_terrain.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
xdino_win64_main.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
xdino_win64_rdr.obj	C:\Users\j.vernay\Documents\CoursProgJV\build\obj\x64-windows\Debug\Dino_JulienVernay
stb.cpp	C:\Users\j.vernay\Documents\CoursProgJV\external\stb
dino_animal.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_draw_utils.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_game.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_geometry.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_player.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
dino_terrain.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino
xdino_linux_main.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-linux
xdino_win64_main.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-windows
xdino_win64_rdr.cpp	C:\Users\j.vernay\Documents\CoursProgJV\src\dino\x64-windows

Fichiers OBJ

build/obj

Chaque SRC a son fichier OBJ

src/

external/

⟹ OBJ → Pour chaque plateforme/configuration

j.vernay CoursProgJV !tools *.exe - Everything

Fichier Edition Affichage Recherche Signets Outils Aide

j.vernay CoursProgJV !tools *.exe

Nom	Chemin
Dino_JulienVernay.exe	C:\Users\j.vernay\Documents\CoursProgJV
Dino_JulienVernay.exe	C:\Users\j.vernay\Documents\CoursProgJV\build\x64-windows\Debug
Dino_JulienVernay.exe	C:\Users\j.vernay\Documents\CoursProgJV\build\x64-windows\Release

Exe de démo

Exe DEBUG

Exe RELEASE

H / .CPP → SRC\ EXTERNAL\

Premake5.lua

```
-- Compile tous les fichiers qui sont directement dans le dossier src
files { "src/dino/*" }
-- Compile tous les fichiers qui sont dans le dossier (et sous-dossiers) external
files { "external/**.cpp", "external/**.h" }
```

.OBJ → BUILD\OBJ\x64-windows\ DEBUG / RELEASE

```
-- Où sont générés les fichiers dy système de build (MSBuild)
location "build"
```

→ Premake crée automatiquement le reste du chemin

.EXE → BUILD\x64-windows\ DEBUG / RELEASE

```
-- Où sont générés les binaires (exécutables et bibliothèques natives)
targetdir "build/%{cfg.platform}/%{cfg.buildcfg}"
```

PRÉPROCESSEUR

COMPILATION (compilateur)

EDITION DES LIENS (linker)

dino-game.cpp → dino-game.obj

dino-animal.h ← dino-animal.cpp → dino-animal.obj

→ Dino.exe

1. Préprocesseur → #include #define #if #endif #pragma

#__ → Directive de préprocesseur

```
#define BEGIN {
#define END }

#define EXPR a+b;

int add(int a, int b)
BEGIN
    return EXPR
END
```

PRÉPROCESSEUR

COMPILATION (compilateur)

EDITION DES LIENS (linker)

dino-game.cpp → dino-game.obj

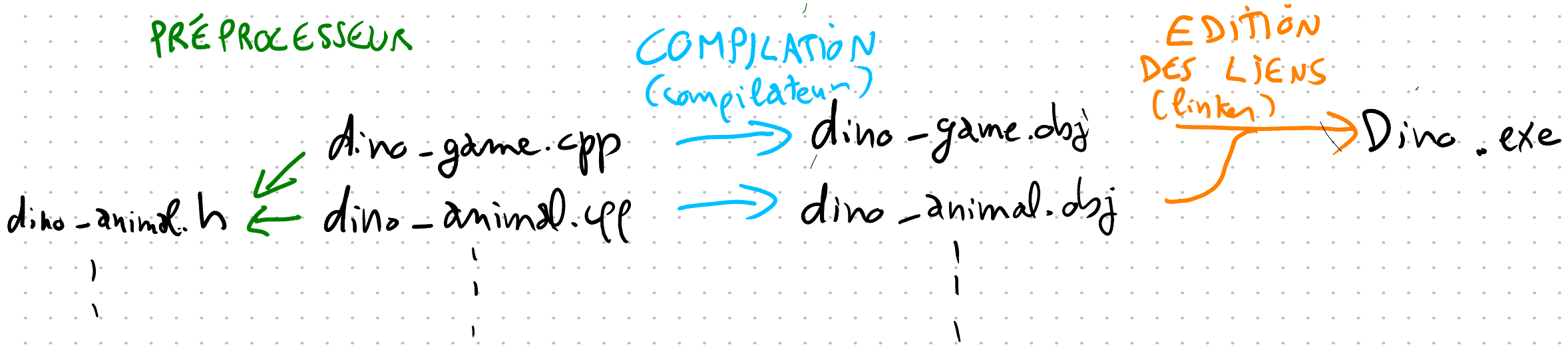
dino-animal.h ← dino-animal.cpp → dino-animal.obj

→ Dino.exe

1. Préprocesseur → #include #define #if #endif #pragma

```
#if XDINO_DEBUG
    Affichage de debug
#endif
#if __cplusplus >= 202001L
    Fonctionnalités C++20
#else
    Fonctionnalité plus vieille
#endif
```

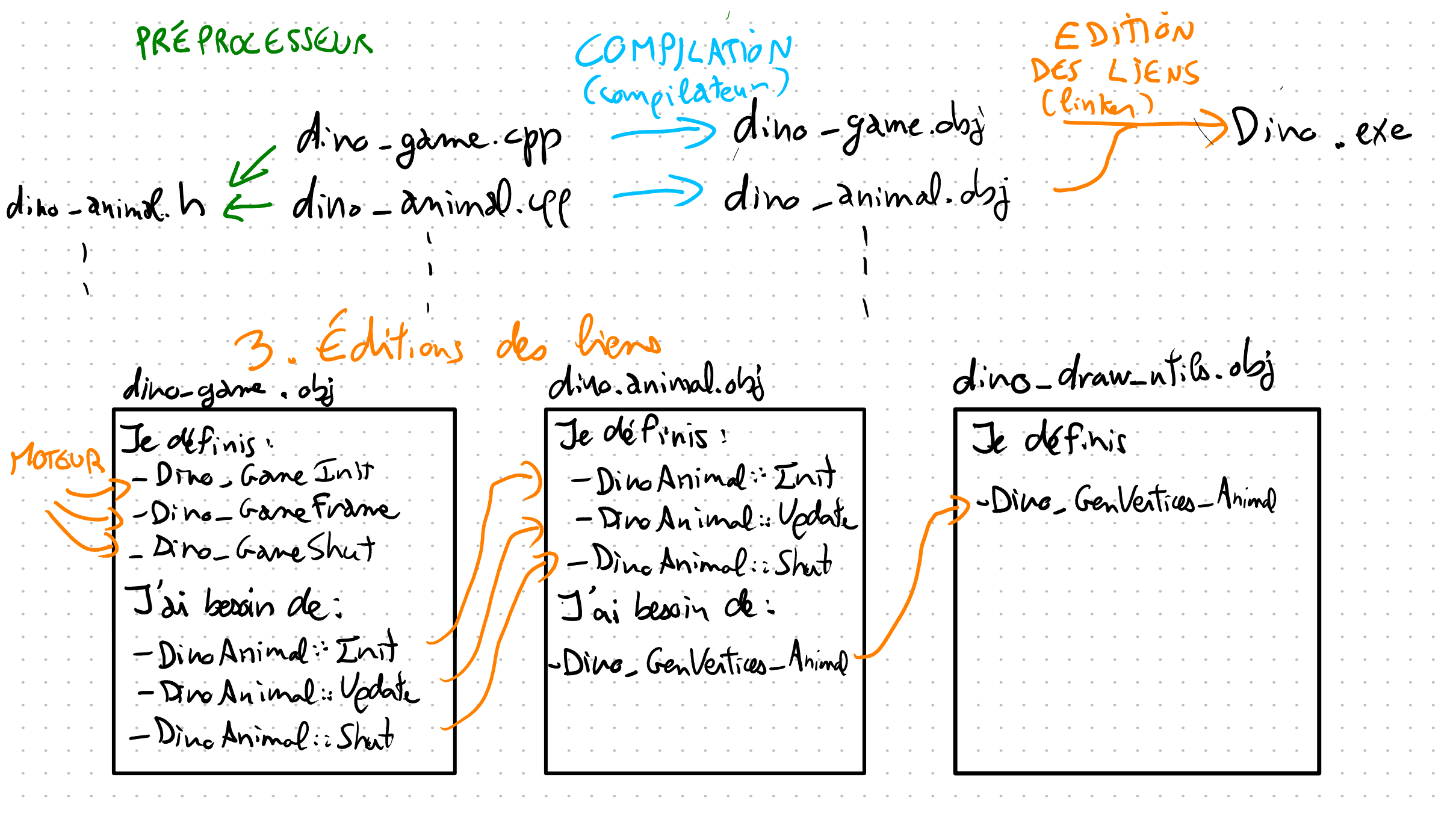
Compilation conditionnelle

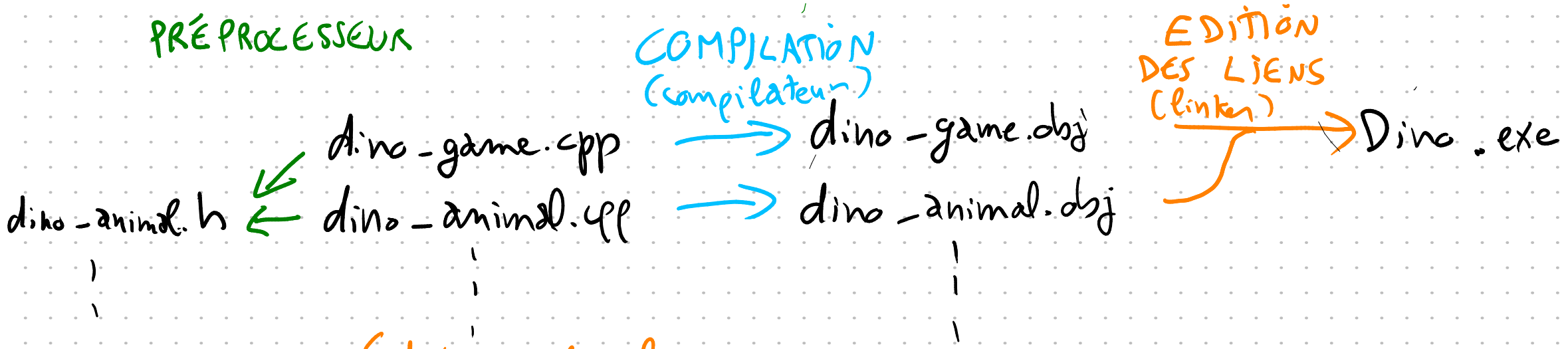


2. Compilation

Le compilateur vérifie les règles du C++ :

- Quelles opérations on peut faire sur quels types
- Éléments de syntaxe
- Transformation du code C++ en code Assembleur
  = code natif


PRÉPROCESSEUR
COMPILATION
(compilateur)
EDITION
DES LIENS
(linker)
dino-game.cpp
dino-animal.h
dino-animal.cpp
dino-game.obj
dino-animal.obj
Dino.exe
3. Éditions des liens
dino-game.obj
MOTEUR
Je définis :
- Dino_GameInit
- Dino_GameFrame
- Dino_GameShut
J'ai besoin de :
- DinoAnimal::Init
- DinoAnimal::Update
- DinoAnimal::Shut
dino.animal.obj
Je définis :
- DinoAnimal::Init
- DinoAnimal::Update
- DinoAnimal::Shut
J'ai besoin de :
- Dino_GenVertices_Animal
dino_draw_utils.obj
Je définis
- Dino_GenVertices_Animal



3 Éditions des liens

→ Faire le lien entre les codes assembleur contenus dans plusieurs fichiers OBJ

Cette slide est bonus : elle ne sera pas à l'examen.

PRÉPROCESSEUR

COMPILATION (compilateur)

EDITION DES LIENS (linker)

dino-game.cpp → dino-game.obj

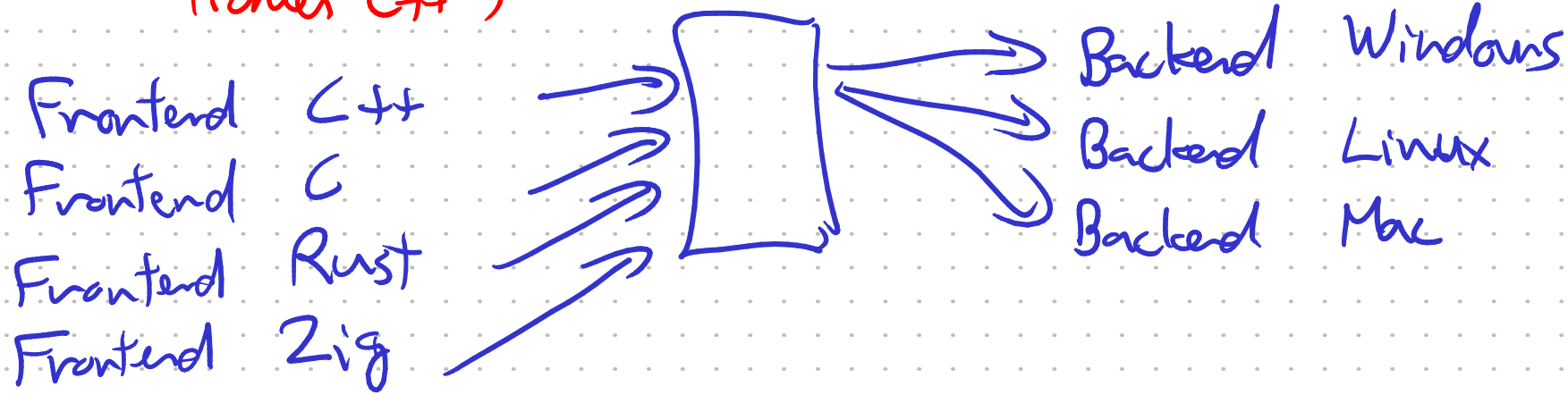
dino-animal.h ← dino-animal.cpp → dino-animal.obj

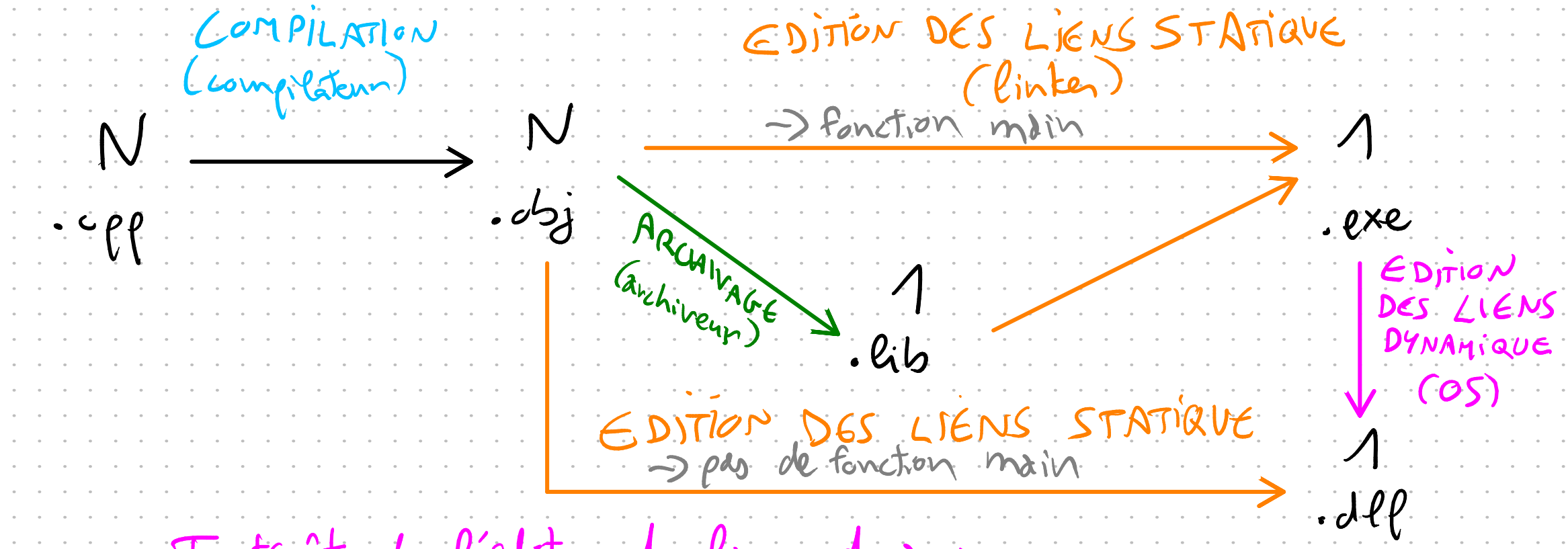
→ Dino.exe

FRONTEND (réussir à lire le fichier C++)

IDE RIDER lit les infos intermédiaires

BACKEND (produire du code natif)



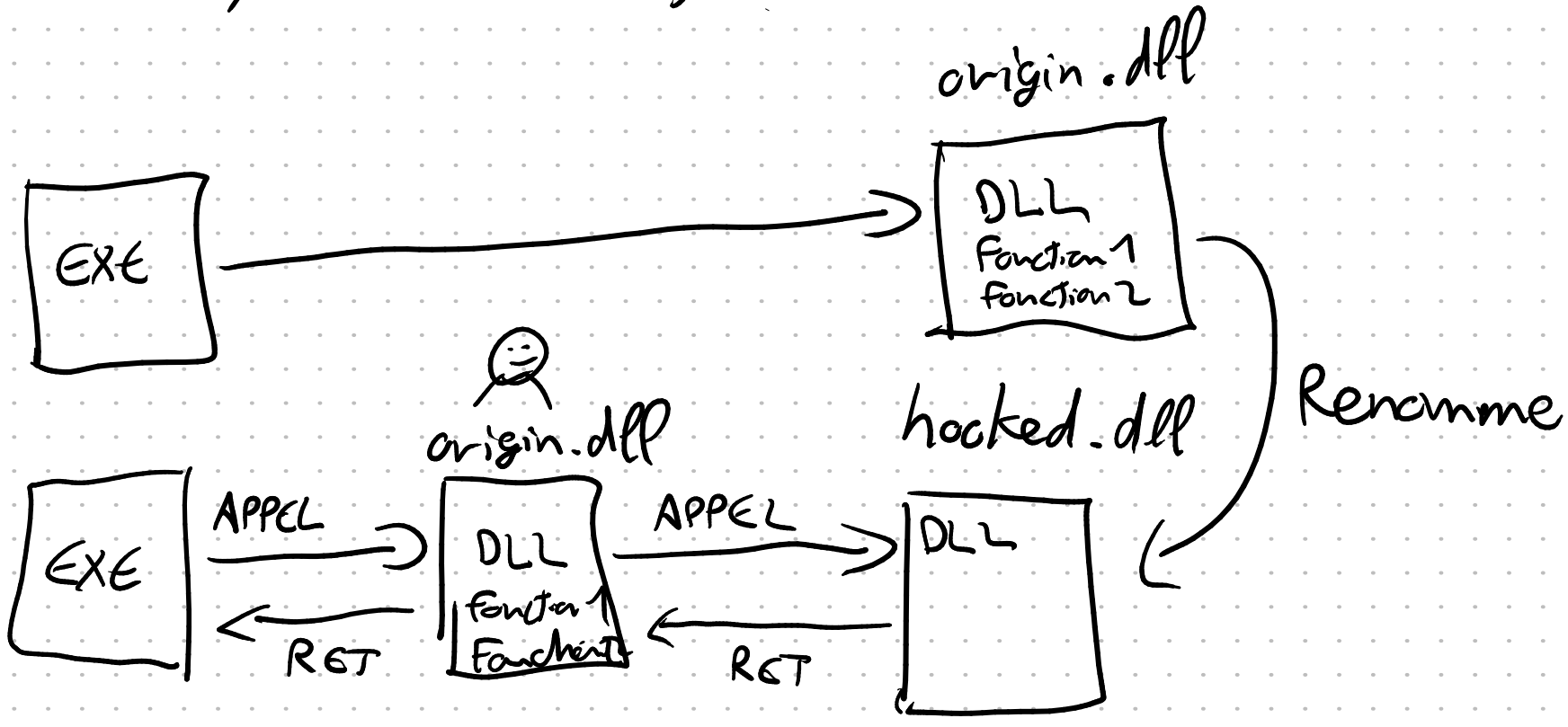


Intérêt de l'édition des liens dynamique :

On peut changer le fichier DLL
sans changer le fichier EXE

Ex : Steam fournit des fonctionnalités (authentification, jeu en ligne, liste d'amis...)
sous forme de DLL (steamclient.dll)

BONUS, À VIRER, MODDING

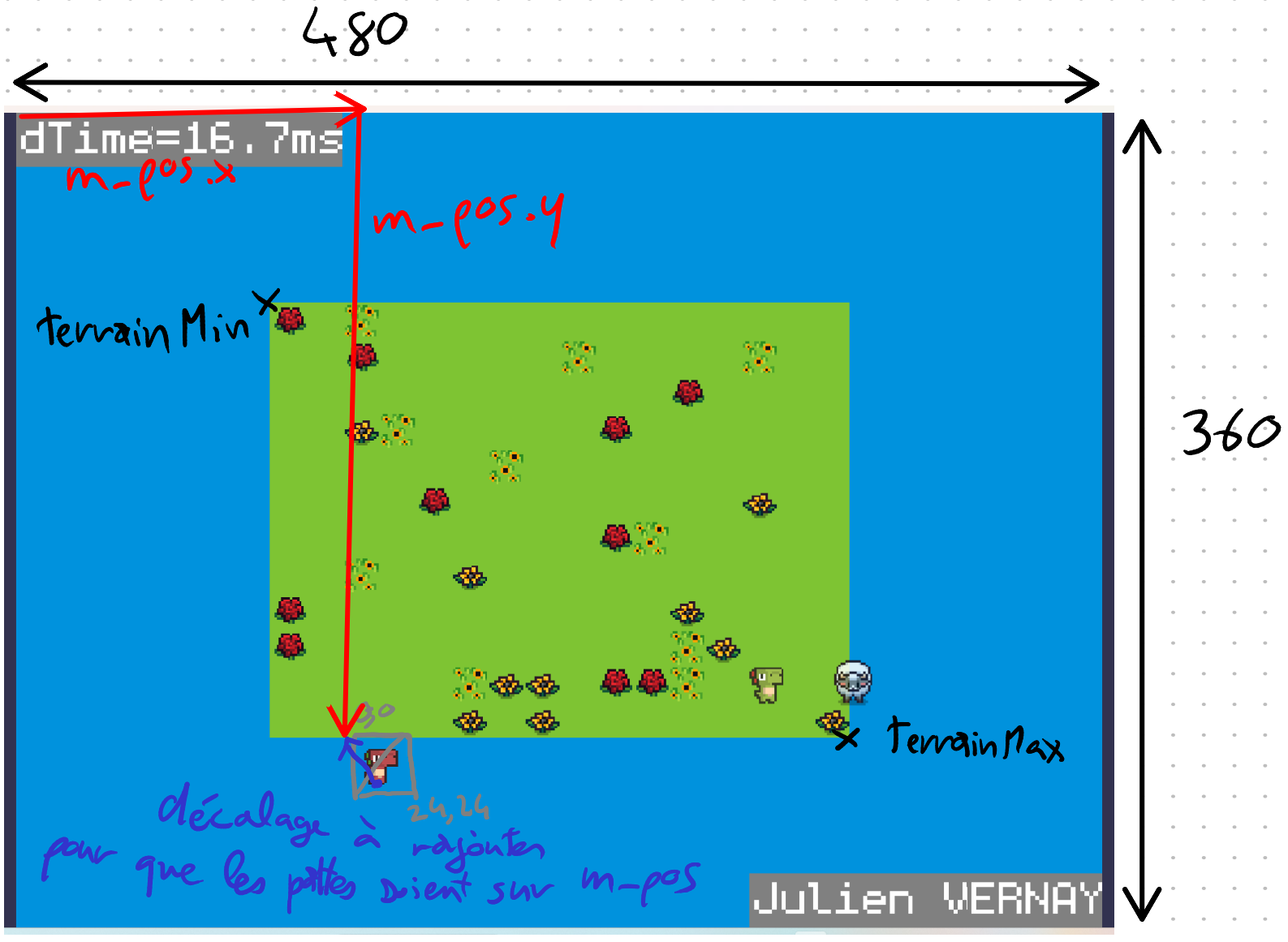


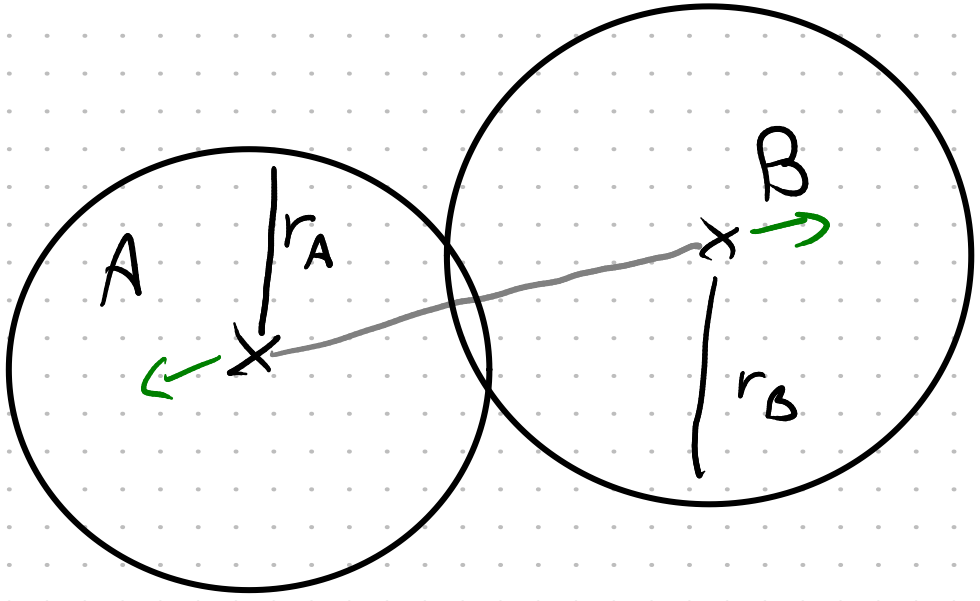
5c)

timeAlive		alpha
0s	$\longrightarrow$	0
1s	$\longrightarrow$	255

$$\text{timeAlive} \longrightarrow \text{alpha} = \frac{\text{timeAlive} \times 255}{1s}$$

(par produit en croix)


480
360
dTime=16.7ms
m_pos.x
m_pos.y
terrainMin
terrainMax
30
24,24
décalage à rajouter
pour que les pattes soient sur m_pos
Julien VERNAY

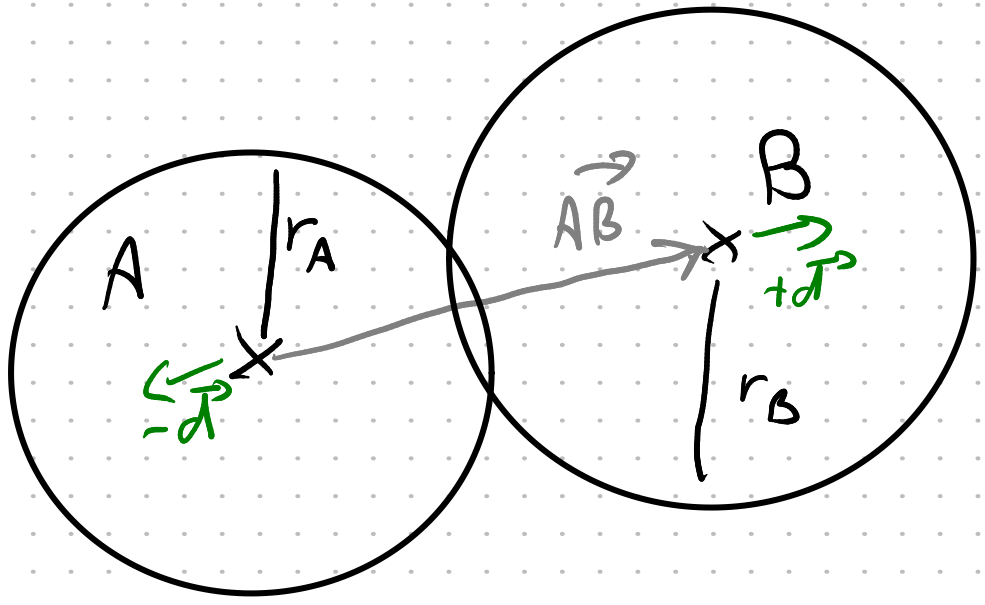


$$AB < r_a + r_b \rightarrow \text{Collision}$$

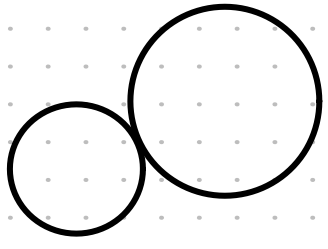
$$\sqrt{(x_B - x_A)^2 + (y_B - y_A)^2} < r_a + r_b \rightarrow \text{Collision}$$

$(r_a = r_b = 8px)$

$\vec{d}$ → Une quantité
→ Une direction



Résolution de collisions



Quantité

$$2d + AB = r_A + r_B$$

$$2d + \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2} = r_a + r_b \iff d = \frac{r_a + r_b - \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}}{2}$$

Direction : $\dfrac{\vec{AB}}{AB}$

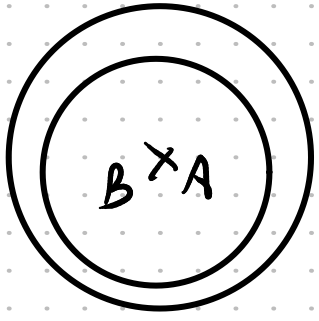
$$\vec{d} = \frac{r_a + r_B - AB}{2\,AB}\,\vec{AB}$$

Risque de division par zéro

$$\vec{d} = \frac{r_A + r_B - AB}{2\,AB}\,\overrightarrow{AB}$$

Risque de division par zéro

$$AB = 0$$

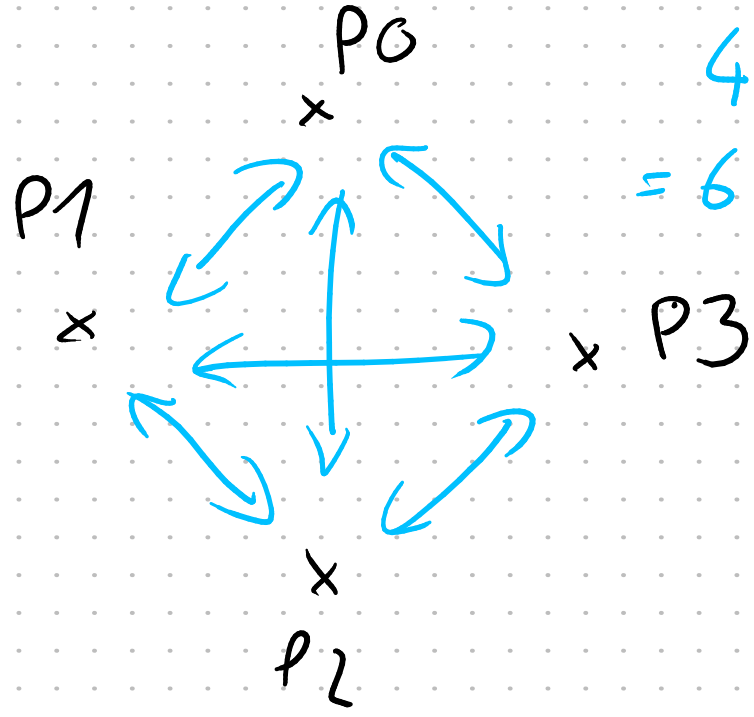


→ Situation très rare (sauf au spawn)

→ Si $A = B$ → Annuler la résolution de collisions

```
for (size_t idxA = 0; idxA < g_Players.size(); ++idxA)
    for (size_t idxB = idxA + 1; idxB < g_Players.size(); ++idxB)
        DinoPlayer::ResolveCollision(g_Players[idxA], g_Players[idxB]);
```

idx A	idx B
0	1
0	2
0	3
1	2
1	3
2	3
3	∅



4 joueurs
= 6 interactions

Héritage en C++

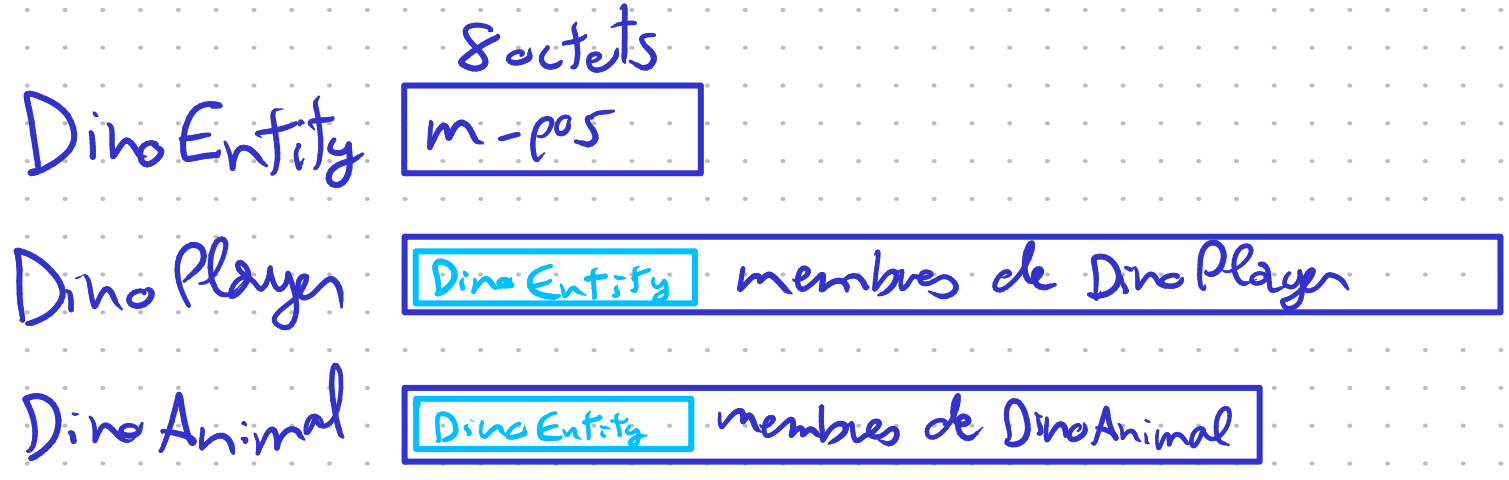
```
class DinoEntity {——};
class DinoPlayer : public DinoEntity {——};
class DinoAnimal : public DinoEntity {——};
```

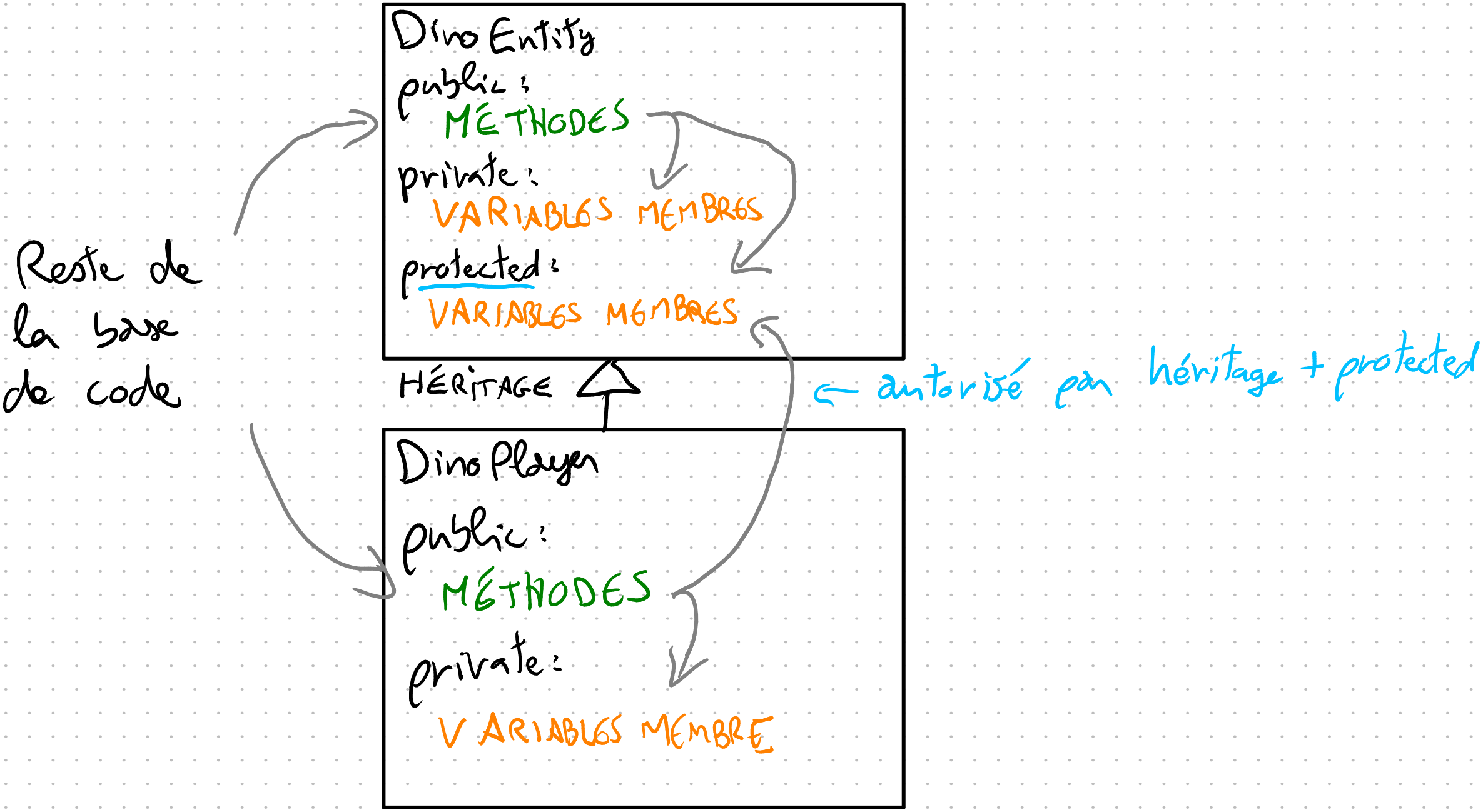
Héritage

Héritage public

(en C++, on peut avoir de l'héritage privé, mais rarement utilisé)

En mémoire RAM :




DinoEntity
public :
MÉTHODES
private :
VARIABLES MEMBRES
protected :
VARIABLES MEMBRES
HÉRITAGE
DinoPlayer
public :
MÉTHODES
private :
VARIABLES MEMBRE
Reste de la base de code
← autorisé par héritage + protected

Héritage en C++

```
class DinoEntity { —— };
class DinoPlayer : public DinoEntity { —— };
class DinoAnimal : public DinoEntity { —— };
```

« DinoPlayer <u>est</u> un DinoEntity »

« DinoAnimal <u>est</u> un DinoEntity »

→ Toutes les fonctions qui acceptent des DinoEntity acceptent aussi des DinoPlayer et DinoAnimal.